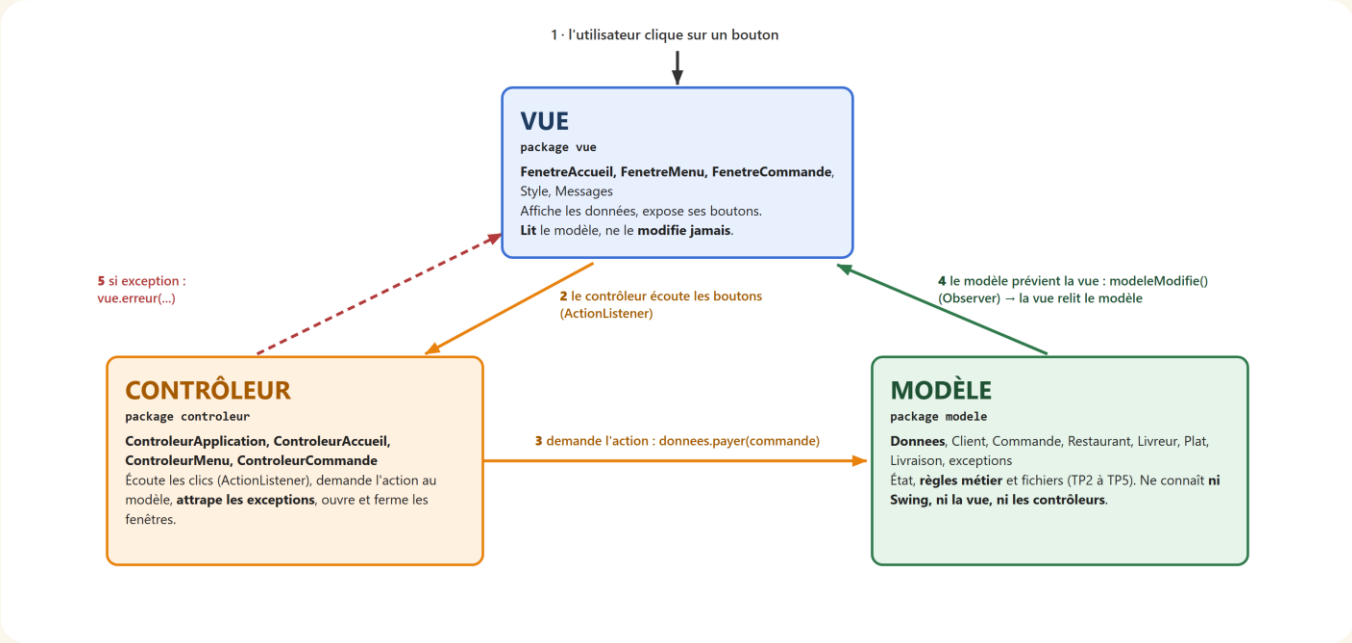
FASTBITE

TP7 — PATTERN MVC (MODÈLE-VUE-CONTRÔLEUR)



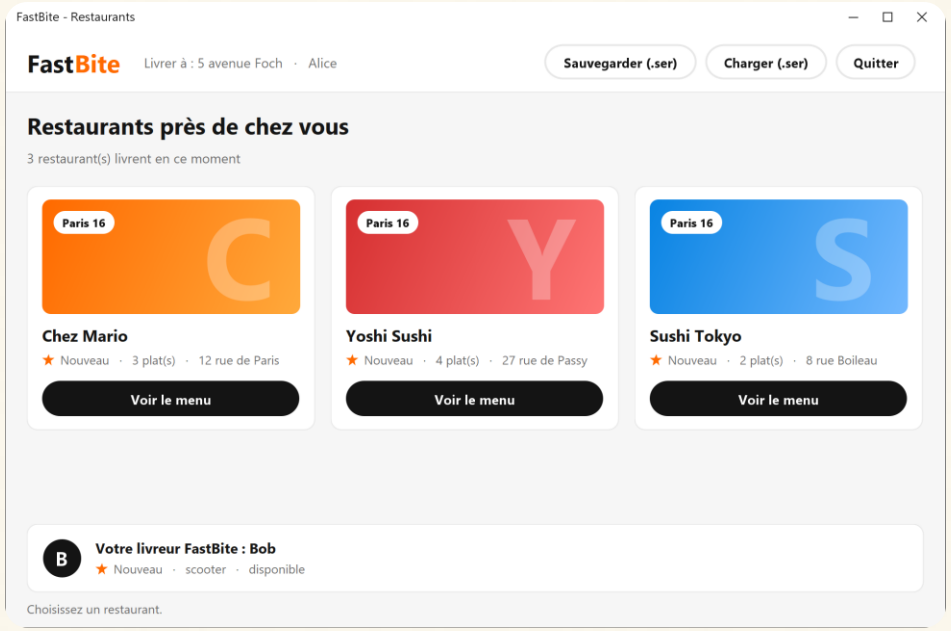
[PRÉNOM NOM 1]
[PRÉNOM NOM 2]
[PRÉNOM NOM 3]

ECE PARIS · ING2 · J.-P. SEGADO

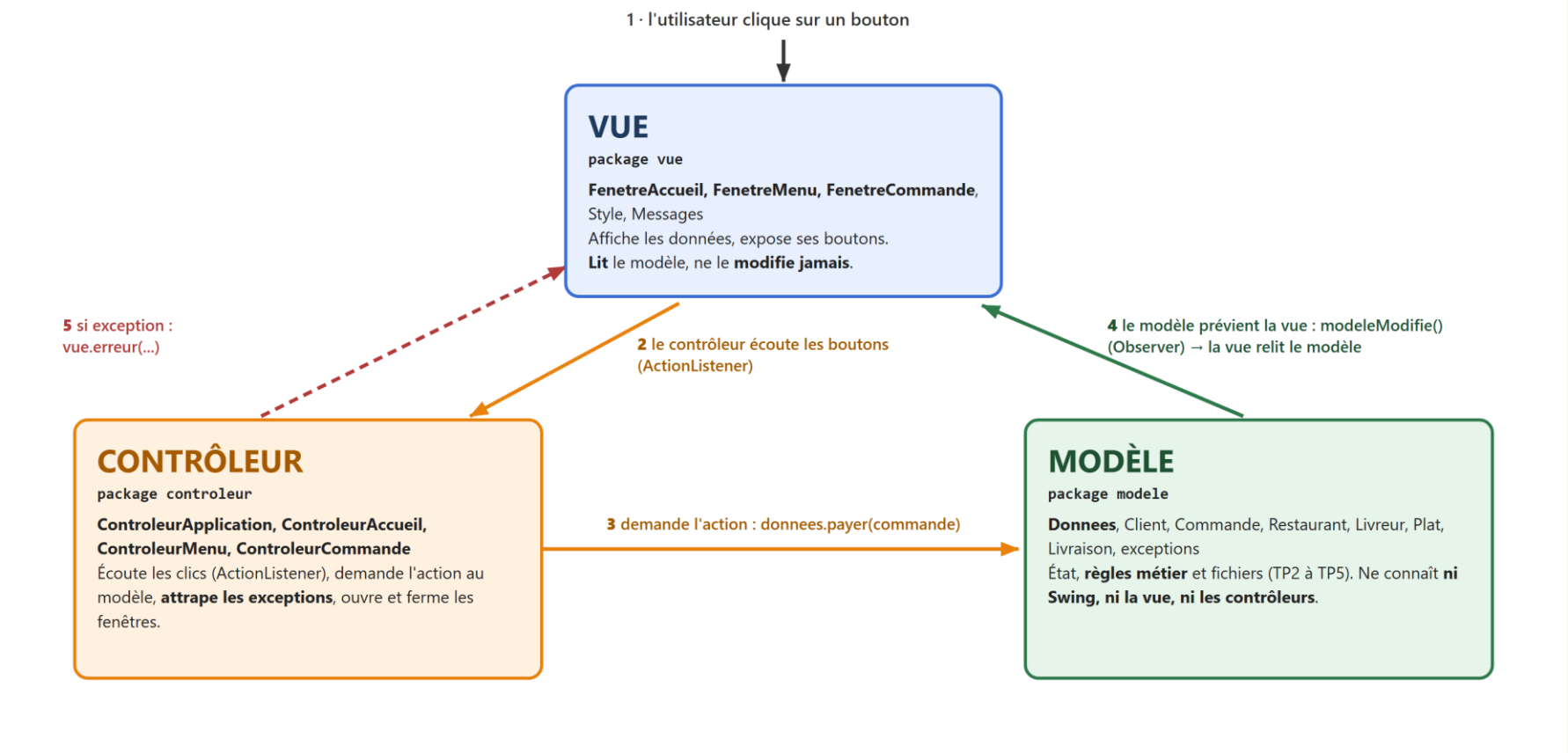
POO-JAVA_2026

SOMMAIRE

- Q1 — LE PATTERN MVC DANS FASTBITE
- AVANT / APRÈS : DU TP6 AU TP7
- MODÈLE, VUE, CONTRÔLEUR : LE CODE
- PREUVES : TESTS AUTOMATIQUES
- BILAN, SOURCES ET USAGE DE L'IA



Q1 — MVC : QUI FAIT QUOI ?



MODÈLE : DONNÉES, RÈGLES MÉTIER ET FICHIERS (TP2 À TP5) — NE CONNAÎT NI SWING NI LA VUE

VUE : FENÊTRES SWING (TP6), LECTURE SEULE DU MODÈLE — CONTRÔLEUR : CLICS, APPELS AU MODÈLE, EXCEPTIONS

AVANT / APRÈS : DU TP6 AU TP7

Responsabilité	TP6 : tout dans la fenêtre	TP7 : MVC
Dessiner les composants (JFrame, layouts)	FenetreXxx	**vue / FenetreXxx (inchangé)**
Écouter les clics (ActionListener)	dans la fenêtre : « implements ActionListener »	**controleur / ControleurXxx**
try / catch des exceptions du TP5	dans la fenêtre	**controleur (la vue ne fait qu'afficher l'erreur)**
Règles métier : livraison après paiement, pourboire, note du livreur	dans la fenêtre (if + message)	**modele / Donnees + 3 nouvelles exceptions**
Créer la commande (le panier)	constructeur de FenetreMenu	**Donnees.nouvelleCommande(), appelée par le contrôleur**
Ouvrir et fermer les autres fenêtres	chaque fenêtre ouvrait la suivante	**controleur / ControleurApplication**
Mettre l'affichage à jour	la fenêtre rappelait rafraichir() à la main	**Observer : le modèle appelle modeleModifie()**
Fichiers .txt et .ser (TP5)	appelés depuis la fenêtre	**Donnees (modèle), demandés par le contrôleur**

MÊME INTERFACE, MÊME COMPORTEMENT : LE CODE EST SIMPLEMENT RANGÉ LÀ OÙ IL DOIT VIVRE

LE MODÈLE : ÉTAT, RÈGLES, OBSERVER

• EcouteurModele.java

```java
public interface EcouteurModele {
    // Appelee par le modele apres chaque modification : la vue relit alors le modele et se redessine
    void modeleModifie();
}
```

• Donnees.java

```java
    // Previent toutes les vues : elles relisent le modele et se redessinent
    public void notifierEcouteurs() {
        for (EcouteurModele e : new ArrayList<EcouteurModele>(ecouteurs)) {
            e.modeleModifie();
        }
    }
```

• Donnees.java

```java
    // Regle metier : pas de pourboire negatif, ni sur une commande deja payee
    public void appliquerPourboire(Commande c, double pourboire) throws PourboireInvalideException {
        if (pourboire < 0) {
            throw new PourboireInvalideException("Le pourboire ne peut pas etre negatif.");
        }
        if (c.getStatut().equals("PAYEE")) {
            throw new PourboireInvalideException("La commande est deja payee : le total ne peut plus changer.");
        }
        c.setPourboire(pourboire);
        notifierEcouteurs();
    }
```

1. Donnees est le point d'entrée du modèle : état (restaurants, client, livreur) et actions métier.

2. EcouteurModele est la seule chose que le modèle connaît des vues : il ne les importe pas.

3. notifierEcouteurs() : après chaque modification, toutes les vues abonnées sont prévenues.

4. La règle « pas de pourboire négatif ni sur une commande payée » est ici, levée en exception (elle était dans la fenêtre au TP6).

5. Aucun import de javax.swing dans le package modele.

LA VUE : ELLE AFFICHE, C'EST TOUT

• FenetreAccueil.java

```java
public class FenetreAccueil extends JFrame implements EcouteurModele {
```

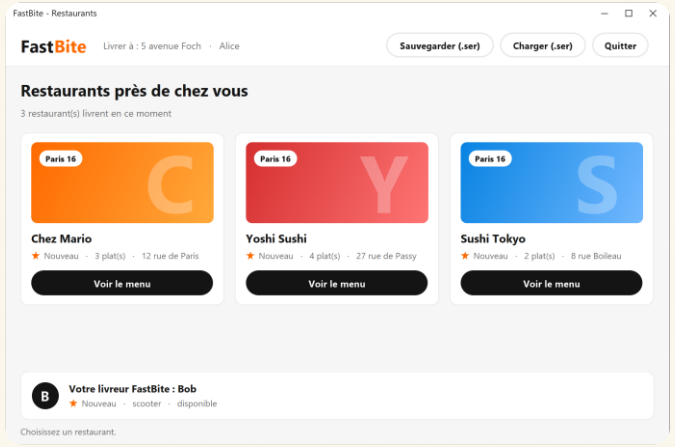
• FenetreAccueil.java

```java
    public void setControleur(ActionListener nouveau) {
        if (controleur != null) {
            for (JButton b : tousLesBoutons()) b.removeActionListener(controleur);
        }
        controleur = nouveau;
        for (JButton b : tousLesBoutons()) b.addActionListener(controleur);
    }
```

• FenetreAccueil.java

```java
    // Observer : le modele a change, la vue se redessine
    @Override
    public void modeleModifie() {
        rafraichir();
    }
```

1. FenetreAccueil implémente EcouteurModele : le modèle la prévient, elle se redessine (modeleModifie).
2. Elle lit Donnees pour s'afficher mais ne le modifie jamais.
3. setControleur(...) : le contrôleur lui est fourni et se branche sur ses boutons, y compris ceux recréés à chaque rafraîchissement.
4. Plus de implements ActionListener, plus de try / catch métier, plus d'ouverture d'autres fenêtres : elle expose seulement ses boutons et erreur(...).



Page 1 :
même fenêtre qu'au TP6

LE CONTRÔLEUR : CLICS, MODÈLE, ERREURS

• ControleurAccueil.java

```java
public void actionPerformed(ActionEvent e) {
    Object source = e.getSource();
    int indice = vue.getBoutonsMenu().indexOf(source);       // -1 si ce n'est pas un bouton « Voir le menu »
    if (indice >= 0) {
        app.ouvrirMenu(indice);
    } else if (source == vue.getBoutonSauver()) {
        sauvegarder();
    } else if (source == vue.getBoutonCharger()) {
        charger();
    } else if (source == vue.getBoutonSuivi()) {
        app.ouvrirSuivi();
    } else if (source == vue.getBoutonQuitter()) {
        vue.fermer();
    }
}
```

• ControleurMenu.java

```java
// Clic « Ajouter » : le modele ajoute le plat et previent la vue, qui redessine le panier
private void ajouterAuPanier(int numero) {
    Plat plat = restaurant.getMenu()[numero];
    try {
        donnees.ajouterPlat(commande, plat);
        vue.setEtat(plat.getNom() + " ajouté au panier.");
    } catch (PlatIndisponibleException ex) {
        vue.erreur("Plat indisponible", ex.getMessage());
    }
}
```

• ControleurCommande.java

```java
private void payer() {
    try {
        donnees.payer(commande);
        vue.setEtat("Paiement accepté : " + Style.prix(commande.calculerTotal()) + ".");
    } catch (PaiementImpossibleException ex) {
        vue.erreur("Paiement refuse", ex.getMessage());
    }
}
```

• Donnees.java

```java
public void payer(Commande c) throws PaiementImpossibleException {
    client.payerCommande(c);
    notifierEcouteurs();
}
```

1. C'est le contrôleur qui implémente ActionListener (plus la fenêtre) ; getSource() dit quel bouton.
2. Il demande l'action au modèle (donnees.ajouterPlat, donnees.payer) puis vue.setEtat(...).
3. Si le modèle refuse, il attrape l'exception et appelle vue.erreur(...).
4. ControleurCommande garde les classes anonymes du TP6.

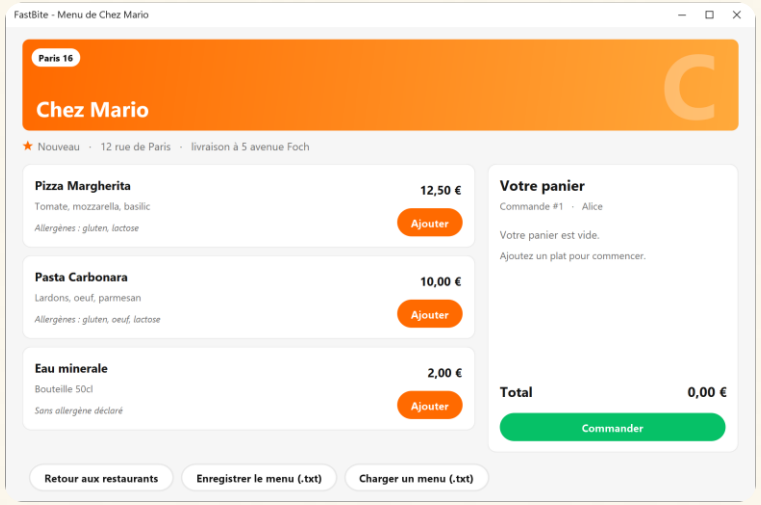
L'ASSEMBLAGE ET LA NAVIGATION

• ControleurApplication.java

```java
// Demarrage : vue + controleur de l'accueil, puis jeu d'essai (le modele previent la vue)
public void demarrer() {
    accueil = new FenetreAccueil(donnees);
    donnees.ajouterEcouteur(accueil);                    // la vue observe le modele
    accueil.setControleur(new ControleurAccueil(accueil, donnees, this));
    try {
        donnees.creerJeuDEssai();
    } catch (CapaciteDepasseeException e) {
        accueil.erreur("Demarrage", e.getMessage());
    }
    accueil.setVisible(true);
}
```

• ControleurApplication.java

```java
public void ouvrirMenu(int indice) {
    Restaurant restaurant = donnees.getRestaurants().get(indice);
    try {
        Commande panier = donnees.nouvelleCommande(restaurant);
        FenetreMenu page = new FenetreMenu(donnees, restaurant, panier, indice);
        menuCourant = new ControleurMenu(page, donnees, this, restaurant, panier, indice);
        page.setControleur(menuCourant);
        donnees.ajouterEcouteur(page);
        page.setVisible(true);
        accueil.setVisible(false);
    } catch (CapaciteDepasseeException ex) {
        // l'historique du client (tableau de 20 commandes) est plein
        accueil.erreur("Commande impossible", ex.getMessage());
    }
}
```



1. FastBiteGUI.main crée le modèle puis le ControleurApplication.
2. demarrer() crée la vue, l'abonne au modèle et branche son contrôleur.
3. ouvrirMenu() : le modèle crée la commande, le contrôleur crée la fenêtre.
4. Plus aucune fenêtre n'en ouvre une autre : toute la navigation est ici.

PREUVES : LE MVC EST RESPECTÉ

Invite de commandes - tests du TP7

```
$ java -cp out;out_tests TestsTP7

--- MVC : la vue observe le modele
ok  OBSERVER : le total de la vue change quand on modifie le modele directement (0,00 EUR -> 10,00 EUR)
--- MVC : les regles metier sont dans le modele
ok  le MODELE refuse la livraison d'une commande non payee (CommandeNonPayeeException)
ok  le MODELE refuse un pourboire negatif (PourboireInvalideException)
ok  le MODELE refuse de noter le livreur avant la livraison (NotationImpossibleException)
--- MVC : controle automatique du code source (src/)
  ok  modele/    : 96 verifications  (aucun javax.swing, java.awt, import vue, import controleur)
  ok  vue/       : 110 verifications  (aucun import controleur, aucune modification du modele)
  ok  controleur/: 24 verifications  (aucun composant graphique construit)
Tous les tests TP7 sont passes (273 verifications).
```

LE TEST LIT NOS FICHIERS SOURCE : LE MODÈLE N'IMPORTE NI SWING NI LA VUE, LA VUE NE MODIFIE PAS LE MODÈLE, LE CONTRÔLEUR NE CONSTRUIT AUCUN COMPOSANT

LES SCÉNARIOS DE CLICS DU TP6 PASSENT TOUJOURS : LE COMPORTEMENT N'A PAS CHANGÉ

BILAN, SOURCES ET USAGE DE L'IA

BILAN

- FAIT : MODÈLE, VUE ET CONTRÔLEURS SÉPARÉS EN 3 PACKAGES, OBSERVER, NAVIGATION CENTRALISÉE
- VÉRIFIÉ : TestsTP7, 273 VÉRIFICATIONS, DONT LE CONTRÔLE DES IMPORTS
- APPRIS : QUI DÉCIDE, QUI AFFICHE, QUI RETIENT LES RÈGLES ; OBSERVER
- LIMITES : LE MODÈLE GARDE DES println DES TP3 ; IL PRÉVIENT LES VUES SANS DIRE CE QUI A CHANGÉ ; Style ATTRAPE AucuneNoteException POUR AFFICHER « NOUVEAU »
- AMÉLIORATIONS : RETIRER LES println DU MODÈLE, ÉVÉNEMENTS PLUS PRÉCIS, TESTS DU MODÈLE SANS FENÊTRE

SOURCES ET IA

COURS DE M. SEGADO	→	TP7 ET COURS 4 À 6 : INTERFACES, SWING
OPENCLASSROOMS, WIKIPÉDIA	→	LE PATTERN MVC
CLAUDE IA	→	REFONTE DU CODE, TESTS, RAPPORT
INTELLIJ	→	COMPILATION ET EXÉCUTION

PART D'AIDE DE L'IA : [XX %] (À COMPLÉTER)

PROMPT : « FAIS LE TP7 (PATTERN MVC) À PARTIR DU TP6 »